

交叉表（crosstab）和透视表（pivot_table）的核心区别

🎯 核心区别总结

维度	<code>pd.crosstab()</code>	<code>df.pivot_table()</code>
主要用途	计算频数/频率（计数）	数据聚合（求和、均值等）
数据来源	直接传入数组/Series	基于 DataFrame
默认聚合	计数（count）	需要指定 <code>aggfunc</code>
灵活性	简单快捷，专注交叉分析	更灵活，支持多种聚合
适用场景	列联表、频率分布	多维度数据分析

📊 一、基本用法对比

1. 创建示例数据

代码块

```
1 import pandas as pd
2 import numpy as np
3
4 # 创建销售数据
5 df = pd.DataFrame({
6     '部门': ['技术', '销售', '技术', '人事', '销售', '技术', '人事', '销售'],
7     '性别': ['男', '女', '男', '女', '男', '女', '男', '女'],
8     '学历': ['本科', '硕士', '硕士', '本科', '本科', '博士', '硕士', '本科'],
```

```
9     '薪资': [8000, 9000, 12000, 7000, 9500, 15000, 7500, 8500],
10     '绩效': [4.5, 3.8, 4.2, 3.5, 4.0, 4.8, 3.2, 3.9]
11 })
12
13 print("原始数据:")
14 print(df)
```

2. 交叉表：统计频数

代码块

```
1  # ===== crosstab: 统计各部门的男女人数 =====
2
3  ct = pd.crosstab(index=df['部门'], columns=df['性别'])
4  print("交叉表 - 各部门男女人数:")
5  print(ct)
```

输出：

代码块

性别	女	男
部门		
人事	1	1
技术	1	2
销售	2	1

3. 透视表：数据聚合

代码块

```

1  # ===== pivot_table: 计算各部门的平均薪资 =====
2
3  pt = df.pivot_table(
4      values='薪资',          # 要聚合的值
5      index='部门',          # 行维度
6      aggfunc='mean'         # 聚合函数
7  )
8  print("透视表 - 各部门平均薪资:")
9  print(pt)

```

输出:

代码块

```

1      薪资
2  部门
3  人事  7250.0
4  技术 11666.67
5  销售  9000.0

```

二、核心功能对比

1. 交叉表的特色功能

代码块

```

1  # ===== 功能1: 计算百分比 (归一化) =====
2
3  ct = pd.crosstab(index=df['部门'], columns=df['性别'])
4  print("原始频数:")
5  print(ct)
6

```

```

7  # 按行归一化（每行加起来=1）
8  ct_row_norm = pd.crosstab(index=df['部门'], columns=df['性别'],
9                             normalize='index')
10 print("\n按行归一化（部门内性别比例）：")
11 print(ct_row_norm.round(3))
12
13 # 按列归一化（每列加起来=1）
14 ct_col_norm = pd.crosstab(index=df['部门'], columns=df['性别'],
15                             normalize='columns')
16 print("\n按列归一化（性别中各部门比例）：")
17 print(ct_col_norm.round(3))
18
19 # 全部归一化（总和=1）
20 ct_all_norm = pd.crosstab(index=df['部门'], columns=df['性别'], normalize='all')
21 print("\n全部归一化（占总人数比例）：")
22 print(ct_all_norm.round(3))

```

代码块

```

1  # ===== 功能2：添加边际总计（小计/合计） =====
2
3  ct_margins = pd.crosstab(
4      index=df['部门'],
5      columns=df['性别'],
6      margins=True,          # 添加总计
7      margins_name='总计'    # 总计的名称
8  )
9  print("带边际总计的交叉表：")
10 print(ct_margins)

```

输出：

代码块

```

1  性别  女  男  总计
2  部门
3  人事   1   1   2
4  技术   1   2   3
5  销售   2   1   3
6  总计   4   4   8

```

```

1  # ===== 功能3: 自定义聚合函数 =====
2
3  # 虽然crosstab主要用于计数, 但也可以指定其他聚合
4  ct_custom = pd.crosstab(
5      index=df['部门'],
6      columns=df['性别'],
7      values=df['薪资'],      # 提供值
8      aggfunc='mean'          # 自定义聚合
9  )
10 print("各部门性别的平均薪资:")
11 print(ct_custom.round(2))

```

2. 透视表的特色功能

代码块

```

1  # ===== 功能1: 多维度聚合 =====
2
3  pt_multi = df.pivot_table(
4      values=['薪资', '绩效'],      # 多个值列
5      index='部门',                  # 行维度
6      columns='性别',                # 列维度
7      aggfunc={'薪资': 'mean', '绩效': 'mean'} # 不同列用不同聚合
8  )
9  print("多维度透视表:")
10 print(pt_multi.round(2))

```

代码块

```

1  # ===== 功能2: 多层索引 (多个行列维度) =====
2
3  pt_hier = df.pivot_table(
4      values='薪资',
5      index=['部门', '学历'],        # 多层行索引
6      columns='性别',
7      aggfunc='mean',
8      fill_value=0                    # 填充缺失值
9  )
10 print("多层索引透视表:")
11 print(pt_hier.round(2))

```

代码块

```
1  # ===== 功能3: 多种聚合函数 =====
2
3  pt_multi_agg = df.pivot_table(
4      values='薪资',
5      index='部门',
6      columns='性别',
7      aggfunc=['mean', 'sum', 'count'] # 同时计算多个统计量
8  )
9  print("多种聚合函数的透视表:")
10 print(pt_multi_agg.round(2))
```

代码块

```
1  # ===== 功能4: 自定义聚合函数 =====
2
3  def salary_range(x):
4      return x.max() - x.min()
5
6  pt_custom = df.pivot_table(
7      values='薪资',
8      index='部门',
9      aggfunc=salary_range # 使用自定义函数
10 )
11 print("各部门薪资极差:")
12 print(pt_custom)
```

三、实际应用场景对比

场景1: 人员结构分析 → 用 crosstab

代码块

```

1  # 问题：各部门的学历分布如何？
2
3  ct_education = pd.crosstab(
4      index=df['部门'],
5      columns=df['学历'],
6      margins=True,
7      margins_name='合计'
8  )
9  print("各部门学历分布:")
10 print(ct_education)
11
12 # 转换为百分比
13 ct_education_pct = pd.crosstab(
14     index=df['部门'],
15     columns=df['学历'],
16     normalize='index'
17 )
18 print("\n各部门学历比例:")
19 print((ct_education_pct * 100).round(1).astype(str) + '%')

```

适用原因：只需要统计人数（频数），不需要对薪资等数值聚合

场景2：销售业绩分析 → 用 pivot_table

代码块

```

1  # 问题：各部门的平均薪资和总薪资是多少？
2
3  pt_salary = df.pivot_table(
4      values='薪资',
5      index='部门',
6      aggfunc=['mean', 'sum', 'count'],
7      margins=True,
8      margins_name='总计'
9  )
10 print("各部门薪资统计:")
11 print(pt_salary.round(2))

```

适用原因：需要对数值进行多种聚合计算（均值、求和、计数）

场景3：复杂多维分析 → 用 pivot_table

代码块

```
1  # 问题：不同部门、性别、学历的薪资情况？
2
3  pt_complex = df.pivot_table(
4      values='薪资',
5      index=['部门', '性别'],
6      columns='学历',
7      aggfunc='mean',
8      fill_value=0
9  )
10 print("多维度薪资分析:")
11 print(pt_complex.round(2))
```

适用原因：需要多层索引和灵活的聚合方式



四、两者可以互相替代的情况

代码块

```
1  # ===== 用 crosstab 实现类似 pivot_table 的功能 =====
2
```

```

3  # 方法1: crosstab 指定 values 和 aggfunc
4  ct_as_pivot = pd.crosstab(
5      index=df['部门'],
6      columns=df['性别'],
7      values=df['薪资'],
8      aggfunc='mean'
9  )
10 print("crosstab 计算平均薪资:")
11 print(ct_as_pivot.round(2))
12
13 # 方法2: pivot_table 实现计数 (类似 crosstab)
14 pt_as_crosstab = df.pivot_table(
15     index='部门',
16     columns='性别',
17     aggfunc='size',      # size 相当于计数
18     fill_value=0
19 )
20 print("\npivot_table 实现计数:")
21 print(pt_as_crosstab)

```

五、选择决策指南

代码块

```

1  如何选择?
2  |
3  |— 只需要统计频数/频率?
4  |   |— 是 → 用 crosstab ✓
5  |   |   • 更简洁
6  |   |   • 内置归一化
7  |   |   • 快速查看分布
8  |   |
9  |   |— 否 ↓
10 |
11 |— 需要对数值聚合 (求和、均值等)?
12 |   |— 是 → 用 pivot_table ✓
13 |   |   • 支持多种聚合
14 |   |   • 更灵活

```

```
15 | |
16 | └─ 否 ↓
17 |
18 └─ 需要多层索引?
19 | └─ 是 → 用 pivot_table ✓
20 | |
21 | └─ 否 ↓
22 |
23 └─ 需要同时多个聚合函数?
24 | └─ 是 → 用 pivot_table ✓
25 | |
26 └─ 否 → 两者都可以, 看个人习惯
```

六、完整实战示例

代码块

```
1 import pandas as pd
2 import numpy as np
3
4 # 创建电商订单数据
5 np.random.seed(42)
6 n = 200
7
8 df_orders = pd.DataFrame({
9     '订单ID': range(1, n+1),
10    '地区': np.random.choice(['华东', '华南', '华北', '西南'], n),
11    '品类': np.random.choice(['电子', '服装', '食品', '家居'], n),
12    '客户等级': np.random.choice(['普通', '银卡', '金卡', '钻石'], n),
13    '订单金额': np.random.randint(50, 5000, n),
14    '数量': np.random.randint(1, 20, n)
15 })
16
17 print("="*70)
18 print("电商数据分析: crosstab vs pivot_table")
19 print("="*70)
20
21 # ===== 分析1: 各地区各品类的订单数量 (频数) =====
```

```

22 print("\n【分析1】各地区各品类的订单数量")
23 print("-"*70)
24
25 ct_orders = pd.crosstab(
26     index=df_orders['地区'],
27     columns=df_orders['品类'],
28     margins=True,
29     margins_name='合计'
30 )
31 print(ct_orders)
32
33 # 查看占比
34 ct_orders_pct = pd.crosstab(
35     index=df_orders['地区'],
36     columns=df_orders['品类'],
37     normalize='index'
38 )
39 print("\n各地区品类占比 (%) :")
40 print((ct_orders_pct * 100).round(1))
41
42 # ===== 分析2: 各地区各品类的平均订单金额 (聚合) =====
43 print("\n\n【分析2】各地区各品类的平均订单金额")
44 print("-"*70)
45
46 pt_amount = df_orders.pivot_table(
47     values='订单金额',
48     index='地区',
49     columns='品类',
50     aggfunc='mean',
51     margins=True,
52     margins_name='平均'
53 )
54 print(pt_amount.round(2))
55
56 # ===== 分析3: 不同客户等级的消费情况 (多维聚合) =====
57 print("\n\n【分析3】不同客户等级的消费统计")
58 print("-"*70)
59
60 pt_customer = df_orders.pivot_table(
61     values=['订单金额', '数量'],
62     index='客户等级',
63     aggfunc={
64         '订单金额': ['mean', 'sum', 'count'],
65         '数量': 'mean'
66     }
67 )
68 print(pt_customer.round(2))

```

```
69
70 # ===== 分析4: 复杂多维分析 =====
71 print("\n\n【分析4】各地区-客户等级的平均订单金额")
72 print("-"*70)
73
74 pt_complex = df_orders.pivot_table(
75     values='订单金额',
76     index=['地区', '客户等级'],
77     columns='品类',
78     aggfunc='mean',
79     fill_value=0
80 )
81 print(pt_complex.round(2))
82
83 print("\n✅ 分析完成! ")
```

 七、关键区别总结表

特性	crosstab	pivot_table
本质	频数统计表	数据透视工具
默认操作	计数	需指定聚合函数
输入	Series/数组	DataFrame
归一化	✅ 内置 <code>normalize</code>	❌ 需手动计算
多层索引	❌ 不支持	✅ 支持
多聚合函数	❌ 单一聚合	✅ 支持多个
灵活性	较低（专注计数）	高（通用聚合）
代码简洁度	★★★★ 更简洁	★★★ 稍复杂
学习曲线	容易上手	需要理解参数
典型应用	列联表、频率分布	商业报表、多维分析

最佳实践建议

1. 优先使用 crosstab 的场景

代码块

```
1  # ✓ 快速查看分类变量的分布关系
2  pd.crosstab(df['部门'], df['性别'])
3
4  # ✓ 需要百分比时
5  pd.crosstab(df['部门'], df['性别'], normalize='index')
6
7  # ✓ 简单的交叉分析
8  pd.crosstab(df['地区'], df['品类'], margins=True)
```

2. 优先使用 pivot_table 的场景

代码块

```
1  # ✓ 需要数值聚合
2  df.pivot_table(values='薪资', index='部门', aggfunc='mean')
3
4  # ✓ 多层分析
5  df.pivot_table(values='销售额', index=['地区', '城市'], columns='季度')
6
7  # ✓ 多种统计量
8  df.pivot_table(values='成绩', index='班级', aggfunc=['mean', 'std'])
```

3. 记忆口诀

- `crosstab` → Count（计数）→ 统计**有多少人**
 - `pivot_table` → Process（处理）→ 计算**多少钱/多少分**
-

总结：

- 如果只是想**看分布和频数**，用 `crosstab` 更快更简洁
- 如果需要对数据进行**各种聚合计算**，用 `pivot_table` 更强大更灵活
- 两者在某些场景下可以互换，选择哪个取决于你的具体需求和个人习惯